GUIA TÉCNICO | 16

Arquiteturas clássicas e transferência de aprendizado

Capítulo 16/20 · Fundamentos, experiência reproduzível e interpretação

PROPÓSITO E OBJETIVOS

- Distinguir extração de características, congelamento e fine-tuning.
- Relacionar a técnica à sua função de perda, representação e protocolo de avaliação.
- Explicar limitações e formular um teste reprodutível.

MAPA DE ESTUDO

1. Fundamentação do método
2. Expressões e hipóteses
3. Experimento essencial executável
4. Laboratório aplicado e limitações
5. Exercício e verificações

01 FUNDAMENTAÇÃO CONCEITUAL

VGG usa blocos sequenciais de convoluções; ResNet usa conexões residuais que facilitam propagação de gradientes. Transfer learning reutiliza representações aprendidas e adapta cabeças. Fine-tuning exige taxa pequena e controle do conjunto de validação.

Fonte histórica de apoio: notas do curso original, consultadas apenas como referência de contexto; as explicações e exemplos desta edição são independentes.

Questões críticas: quais pressupostos são necessários? O que a função aprende? Qual métrica indica utilidade real? Que tipo de vazamento ou má interpretação invalidaria a conclusão?

02 MATEMÁTICA E DIMENSÕES

Expressão esquemática: h_(l+1)=F(h_l)+h_l

A expressão acima resume a relação central. Para a derivação, identifique dimensões, hipóteses e operações; não interprete notação compacta como uma implementação completa.

LEITURA CRÍTICA DO CAPÍTULO

Transfer learning exige pesos pré-treinados e alinhamento de pré-processamento; congelar pesos aleatórios não é transferência.

Procedimento: execute a próxima célula com o núcleo declarado em requirements.txt, confira os assert e interprete os resultados. O experimento é independente dos exemplos anteriores.

03 EXPERIMENTO ESSENCIAL

Código extraído do notebook revisado. Execute a célula completa no JupyterLab; as quebras visuais abaixo não substituem o arquivo executável.

```
import numpy as np
rng=np.random.default_rng(5)
x=rng.normal(size=(3,4));W=rng.normal(size=(4,4))*.1
res=x+np.tanh(x@W)
assert res.shape==x.shape
print('Bloco residual (demonstração de shapes):',x.shape,'→',res.shape)
```

INTERPRETAÇÃO

Compare o resultado obtido com a formulação matemática. Confirme shapes, escala e hipótese de modelagem. Um experimento sintético valida uma operação, não substitui avaliação em dados reais. Ao treinar modelos, separe dados e ajuste transformações apenas no treino; preserve um conjunto final de teste.

04 LABORATÓRIO APLICADO

Laboratório aplicado revisado — Extração de características e distinção de transferência real

Transfer learning exige pesos pré-treinados e alinhamento de pré-processamento; congelar pesos aleatórios não é transferência.

Procedimento: execute a próxima célula com o núcleo declarado em requirements.txt, confira os assert e interprete os resultados. O experimento é independente dos exemplos anteriores.

```
import numpy as np
rng=np.random.default_rng(16)
features=rng.normal(size=(25,6))
fixed=rng.normal(size=(6,4))
representation=np.maximum(features@fixed,0)
print('Representação fixa (demonstração):',representation.shape)
print('ATENÇÃO: matriz aleatória não é backbone pré-treinado; não constitui transfer  ↪
    learning.')
assert representation.shape==(25,4)
```

Transferência requer pesos aprendidos previamente em outra tarefa. Um backbone inicializado aleatoriamente e congelado NÃO transfere conhecimento; documentar fonte, preprocessamento e licença dos pesos.

Checklist: conferir entradas e saídas, sementes, inexistência de uso dos dados de teste durante ajuste e distinção entre uma demonstração sintética e desempenho em aplicações reais.

05 EXERCÍCIO E SOLUÇÃO COMENTADA

Por que congelar inicialmente o backbone pré-treinado?

Resposta: Para adaptar a cabeça com baixo custo e reduzir risco de destruir representações úteis com gradientes grandes.

LIMITAÇÕES E BOAS PRÁTICAS

- Confundir ajuste no treino com generalização para dados não vistos.
- Reportar somente uma métrica ou uma execução aleatória.
- Ignorar shape, tipo de variável e convenção do backend.
- Executar exemplos com pesos baixados sem registrar versão, licença e procedência.
- Confundir demonstração matemática com validação de um modelo de produção.

VERIFICAÇÃO LOCAL

Execute Reiniciar kernel e executar todas as células. Registre versões do Python e das dependências,

confira shapes e métricas e diferencie os laboratórios opcionais dos exemplos NumPy/scikit-learn.

Validação posterior (13/11/2026): notebooks 14 a 20 executados com TensorFlow CPU; opcionais 01 a 13 não integralmente revalidados.

REFERÊNCIAS TÉCNICAS

NumPy: numpy.org/doc · scikit-learn: scikit-learn.org/stable/user_guide.html · Keras: keras.io/guides · TensorFlow: tensorflow.org/guide